Мультимодальная система планирования перевозки животных на основе Vision-Language моделей

Vasily Lavrentyev

4 марта 2026 г.

Аннотация

В данной работе рассматривается задача автоматизированного планирования перевозки животных на основе анализа изображения и географических данных маршрута. Пользователь предоставляет фотографию животного, а также начальную и конечную точки перевозки. На основе этих данных система должна определить возможные способы транспортировки и оценить приблизительное время доставки.

Предлагается мультимодальная архитектура, объединяющая Vision-Language модель для извлечения физических характеристик животного, географический модуль для анализа маршрута и модуль правил для определения допустимых видов транспорта. Для извлечения характеристик используется Vision-Language модель Qwen2-VL, дообученная с применением LoRA-адаптации. Инференс модели выполняется локально с использованием фреймворка vLLM.

В результате система формирует структурированный JSON-ответ, содержащий физические параметры животного, географические характеристики маршрута, допустимые виды транспорта и оценку времени перевозки. Предложенный подход демонстрирует возможность использования компактных мультимодальных моделей для решения прикладных задач логистического планирования.

1 Введение

Задача автоматизированного планирования перевозки животных представляет собой междисциплинарную проблему, находящуюся на пересечении компьютерного зрения, систем маршрутизации и больших языковых моделей. В общем виде система должна по входному изображению животного и заданным географическим точкам определить возможные способы транспортировки и приблизительное время доставки. Такая постановка задачи требует объединения нескольких компонентов: визуального распознавания животного, извлечения семантических признаков (например, вида или размера), а также логического вывода на основе транспортной инфраструктуры.

Классические системы планирования маршрутов основаны на графовых моделях транспортных сетей. Например, движок OSRM (Open Source Routing Machine) позволяет выполнять высокопроизводительный поиск маршрутов на основе данных OpenStreetMap [1]. Для мультимодального транспорта используются системы наподобие OpenTripPlanner, которые объединяют дорожные графы и расписания общественного транспорта в формате GTFS (General Transit Feed Specification) [2, 3]. Эти технологии широко применяются в системах навигации и планирования поездок.

Однако в рассматриваемой задаче важным отличием является необходимость обработки визуальной информации. Современные методы компьютерного зрения позволяют автоматически определять тип животного по изображению. Многие подходы используют сверточные нейронные сети и transfer learning на базе датасетов животных, например ImageNet или специализированных коллекций изображений животных [4]. Для задач более точной классификации существуют специализированные датасеты, такие как Stanford Dogs, содержащий более 20 000 изображений собак 120 пород [5]. Также применяются более простые наборы данных для многоклассовой классификации животных, включающие изображения различных видов животных [6].

В последние годы активно развивается направление мультимодальных моделей, объединяющих обработку текста и изображений. Vision–Language Models (VLM) объединяют визуальный энкодер (например, CLIP или ViT) с языковой моделью и позволяют выполнять рассуждения на основе визуальных данных [7]. Одним из наиболее известных примеров является архитектура LLaVA, в которой визуальные признаки проецируются в пространство токенов языковой модели, позволяя выполнять совместное мультимодальное рассуждение [8]. Подобные модели способны извлекать из изображения семантические признаки и использовать их для дальнейшего логического вывода.

Особый интерес представляют компактные мультимодальные модели, пригодные для локального развертывания. Исследования показывают, что даже относительно небольшие модели (несколько миллиардов параметров) способны эффективно решать мультимодальные задачи, если используются подходящие методы обучения и архитектуры [9, 10]. Это делает возможным создание автономных систем, работающих локально без обращения к облачным сервисам.

Таким образом, задача автоматического определения способа перевозки животного может быть решена посредством интеграции методов компьютерного зрения, мультимодальных языковых моделей и систем маршрутизации. В данной работе рассматривается архитектура такой системы и анализируются возможные подходы к её реализации.

2 Обзор существующих решений

Исследования, связанные с рассматриваемой задачей, можно условно разделить на три направления: методы распознавания животных на изображениях, мультимодальные языковые модели и системы маршрутизации.

2.1 Распознавание животных на изображениях

Задача классификации животных по изображению является классической задачей компьютерного зрения. Одним из фундаментальных ресурсов для обучения моделей является датасет ImageNet, содержащий миллионы размеченных изображений различных объектов и животных [4]. На его основе разработано множество моделей, демонстрирующих высокую точность распознавания.

Для более специализированных задач используются узкоспециализированные наборы данных. Например, Stanford Dogs представляет собой датасет для fine-grained классификации пород собак и содержит более 20 000 изображений 120 различных пород [5]. Также существуют более компактные датасеты для задач классификации животных, включающие изображения различных видов животных и применяемые для обучения и тестирования

моделей компьютерного зрения [6].

Современные подходы часто используют transfer learning — адаптацию предварительно обученных моделей компьютерного зрения для решения специализированных задач. Такие методы позволяют достигать высокой точности даже при ограниченном объёме обучающих данных [11].

2.2 Мультимодальные языковые модели

В последние годы широкое распространение получили Vision–Language Models (VLM), которые объединяют обработку изображений и текста. Одним из ключевых направлений является интеграция визуального энкодера с языковой моделью, что позволяет выполнять рассуждение на основе визуальной информации.

Одним из наиболее известных представителей данного класса является модель LLaVA, которая соединяет визуальный энкодер CLIP с языковой моделью и позволяет выполнять визуально-текстовые задачи, такие как visual question answering и описание изображений [8]. Архитектура LLaVA основана на проекции визуальных признаков в пространство токенов языковой модели и последующей обработке с помощью механизма внимания.

Последующие исследования показали, что подобные архитектуры могут быть эффективно масштабированы в сторону компактных моделей. Например, LLaVA-Phi демонстрирует возможность построения мультимодальных ассистентов на базе небольших языковых моделей [9]. Аналогично, фреймворк TinyLLaVA исследует влияние различных компонентов архитектуры на производительность компактных мультимодальных моделей и показывает, что небольшие модели могут достигать сопоставимых результатов при правильной организации обучения [10].

2.3 Системы маршрутизации

Планирование маршрутов является одной из ключевых задач транспортных информационных систем. Одним из наиболее широко используемых инструментов является Open Source Routing Machine (OSRM), предназначенный для быстрого поиска маршрутов на основе дорожных графов OpenStreetMap [1].

Для мультимодального планирования маршрутов применяется OpenTripPlanner — система, способная объединять различные виды транспорта, включая пешие маршруты, велосипеды и общественный транспорт [2]. В таких системах широко используется стандарт GTFS, который определяет формат представления расписаний общественного транспорта [3].

Интеграция систем маршрутизации с мультимодальными моделями открывает возможность создания интеллектуальных систем планирования перевозок, которые учитывают не только географические данные, но и семантическую информацию о перевозимом объекте. Такой подход может быть применён для автоматизированного выбора способа транспортировки животных на основе их характеристик, извлечённых из изображения.

3 Описание базового решения

3.1 Постановка задачи

Целью данного проекта является разработка сервиса, способного определить возможные способы перевозки животного между двумя заданными географическими точками и оценить примерное время перевозки. Входными данными сервиса являются:

- изображение животного,
- начальная точка маршрута,
- конечная точка маршрута.

Выходом системы является структурированный ответ, содержащий список доступных видов транспорта (например, автомобиль, поезд, самолёт) и оценку времени перевозки для каждого из них.

В качестве baseline решения была реализована модульная архитектура, состоящая из нескольких специализированных агентов.

3.2 Архитектура системы

Общая архитектура системы состоит из четырёх агентов и оркестратора, координирующего их взаимодействие:

- **Perception Agent** — определяет тип животного по изображению.
- **Geo Agent** — вычисляет расстояние между географическими точками.
- **Routing Agent** — оценивает время перевозки для различных видов транспорта.
- **Policy Agent** — определяет допустимые способы перевозки на основе правил и вывода языковой модели.
- **Orchestrator** — объединяет результаты всех агентов и формирует итоговый ответ.

Архитектура реализована в виде последовательного конвейера, где выход одного агента служит входом для следующего.

3.3 Модуль распознавания

Задачей Perception Agent является определение типа животного на основе изображения. Для этого используется предварительно обученная мультимодальная модель CLIP (Contrastive Language–Image Pretraining) архитектуры ViT-B/32.

CLIP обучена сопоставлять изображения и текстовые описания в общем векторном пространстве. Пусть I — изображение, а T_i — текстовые описания возможных классов. Тогда модель вычисляет эмбеддинги:

$$z_I = f(I), \quad z_{T_i} = g(T_i)$$

где f и g — нейронные сети-энкодеры изображения и текста соответственно.

Класс определяется как текстовое описание, максимизирующее косинусное сходство:

$$\hat{i} = \arg\max_i \frac{z_I \cdot z_{T_i}}{\|z_I\| \|z_{T_i}\|}$$

В baseline системе используется фиксированный набор классов животных (например, dog, cat, bird и др.), достаточный для демонстрации работы сервиса.

Данный агент не требует дополнительного обучения, так как используется предобученная модель.

3.4 Географический модуль

Geo Agent определяет географическое расстояние между двумя заданными точками. Координаты городов извлекаются с помощью геокодирования, после чего расстояние вычисляется по формуле гаверсинусов:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

где:

- R — радиус Земли,
- ϕ — широта,
- λ — долгота.

Результатом работы агента является расстояние между точками в километрах.

3.5 Модуль оценки времени перевозки

Routing Agent оценивает примерное время перевозки на основе расстояния и типичных скоростей различных видов транспорта. Следует отметить, что в данной работе используется упрощённая модель времени перевозки, основанная на средних скоростях транспорта, без учёта расписаний и пересадок.

Используются следующие средние скорости:

- автомобиль: $v_{car} = 70$ км/ч,
- поезд: $v_{train} = 120$ км/ч,
- самолёт: $v_{plane} = 800$ км/ч.

Время перевозки вычисляется как:

$$t = \frac{d}{v}$$

Для воздушного транспорта добавляется дополнительное фиксированное время, моделирующее процедуры регистрации и погрузки.

Данный агент является полностью детерминированным и не использует машинное обучение.

3.6 Модуль принятия решений

Policy Agent отвечает за определение допустимых способов перевозки животных на основе нормативных правил и логического вывода языковой модели.

Для этого используется подход Retrieval-Augmented Generation (RAG). Набор текстовых правил перевозки животных индексируется с помощью модели эмбеддингов Sentence-Transformers.

Каждое правило представляется вектором:

$$e_i = f(text_i)$$

где f — embedding модель.

Для заданного запроса q вычисляется embedding:

$$e_q = f(q)$$

и выполняется поиск ближайших правил с помощью FAISS.

Найденные правила передаются языковой модели Mistral-7B-Instruct, работающей локально через inference-движок vLLM.

Модель получает на вход:

- тип животного,
- расстояние,
- оценку времени перевозки,
- найденные правила.

и формирует структурированный JSON-ответ.

3.7 Используемая язяковая модель

В baseline используется языковая модель Mistral-7B-Instruct с открытой лицензией Apache-2.0.

Модель содержит 7 миллиардов параметров и развернута локально на GPU с использованием vLLM inference engine.

Для уменьшения потребления видеопамяти используется квантованная версия модели (AWQ).

Модель используется только на этапе inference и не требует дополнительного обучения в baseline.

3.8 Модуль координации компонентов

Orchestrator объединяет работу всех агентов:

1. Perception Agent определяет тип животного.
2. Geo Agent вычисляет расстояние.
3. Routing Agent оценивает время перевозки.
4. Policy Agent определяет допустимые способы перевозки.
5. формируется итоговый JSON-ответ.

Таким образом, система реализует модульную multi-agent архитектуру.

3.9 Особенности реализации

Baseline реализован на языке Python.

Используемые библиотеки:

- PyTorch
- open-clip
- sentence-transformers
- FAISS
- geopy
- vLLM

Языковая модель развернута на удалённом GPU сервере, остальные компоненты выполняются локально.

3.10 Характеристики базового решения

Baseline обладает следующими свойствами:

- полностью локальный inference языковой модели,
- отсутствие дополнительного обучения моделей,
- модульная multi-agent архитектура,
- использование retrieval-augmented generation,
- поддержка multimodal входных данных (изображение + текст).

Данный baseline служит отправной точкой для дальнейших улучшений, включая обучение моделей, расширение базы правил и более точное моделирование транспортной доступности.

4 Подход на основе Vision-Language модели

4.1 Подготовка и обогащение обучающего датасета

Для обучения мультимодальной модели, предсказывающей транспортно-значимые характеристики животных по изображению, был сформирован объединённый датасет из нескольких открытых источников. Основная цель подготовки данных заключалась в создании обучающей выборки, покрывающей максимально широкий спектр животных (домашние животные, птицы, насекомые и др.), при этом содержащей только те признаки, которые непосредственно влияют на условия перевозки.

4.1.1 Использованные источники данных

В итоговый набор данных были включены изображения из следующих открытых датасетов:

- **Oxford-IIIT Pet Dataset** — специализированный датасет изображений домашних животных (кошек и собак), содержащий 37 пород. Датасет предложен в работе [12]. Он широко используется в задачах компьютерного зрения и предоставляет надёжные метки пород.
- **Animals-10 Dataset** — открытый набор изображений животных, содержащий 10 категорий (например, dog, cat, cow, horse, spider, butterfly и др.). Датасет доступен на платформе Kaggle [13] и используется для задач классификации животных.

Использование этих двух источников позволило сформировать выборку, содержащую как домашних животных, так и дикие виды, включая насекомых и крупных млекопитающих.

4.1.2 Нормализация структуры данных

Исходные датасеты имеют различную структуру аннотаций. Для обеспечения совместимости данных была выполнена процедура унификации формата аннотаций. Все данные были приведены к формату `JSONL`, где каждая строка представляет отдельный обучающий пример.

Первоначально для каждого изображения формировалась расширенная структура аннотаций, содержащая таксономические признаки и физические характеристики животного. Однако на этапе подготовки финального обучающего набора было принято решение удалить таксономические поля (вид, род, семейство, породу и др.). Это сделано для того, чтобы модель обучалась извлекать исключительно физические характеристики, непосредственно влияющие на условия перевозки, и не опиралась на категориальные подсказки.

В результате в финальный датасет были включены только следующие целевые признаки:

- `size_class` — класс размера животного;
- `weight_class` — приблизительный диапазон массы;
- `brachycephalic` — признак брахицефального строения черепа;
- `needs_carrier` — необходимость использования переноски при транспортировке.

Формат одного обучающего примера имеет вид:

```
{
  "image": "path/to/image.jpg",
  "output": {
    "size_class": "small",
    "weight_class": "<1kg",
    "brachycephalic": false,
    "needs_carrier": "yes"
  }
}
```

4.1.3 Обогащение данных

Поскольку исходные датасеты не содержат всех требуемых транспортно-значимых характеристик, был реализован этап обогащения данных (data enrichment). Для этого использовалась таблица соответствия атрибутов (`attribute_lookup.json`), содержащая сопоставление классов или пород животных с типичными физическими характеристиками.

Процедура обогащения выполнялась автоматически с использованием специально разработанного скрипта. В зависимости от источника данных применялись различные правила:

- **Animals-10.** Для каждой категории животных (например, dog, cow, spider) из справочной таблицы извлекались значения: `size_class`, `weight_class` и `brachycephalic`. Дополнительно применялось правило определения необходимости переноски (`needs_carrier`) на основе заранее заданной таблицы соответствий.
- **Oxford-IIIT Pet.** Для изображений домашних животных использовалась таблица соответствия пород (`oxford_pets_breed`), содержащая информацию о размере животного, диапазоне веса и признаке брахицефалии. По названию породы выполнялось сопоставление и добавление соответствующих атрибутов.

После применения правил обогащения структура атрибутов была дополнительно нормализована: из всех полей удалялись вспомогательные параметры (например, значения уверенности), оставляя только итоговые значения признаков. Такой подход позволяет автоматически генерировать обучающие аннотации без ручной разметки, что значительно снижает стоимость подготовки обучающих данных.

4.1.4 Формирование итогового обучающего набора

После объединения и обогащения данных был сформирован единый датасет. Далее он был разделён на обучающую и валидационную части:

- обучающая выборка: **31 890** изображений;
- валидационная выборка: **1 679** изображений.

Данные хранятся в формате `JSONL`, что позволяет эффективно загружать их при обучении мультимодальных моделей.

4.1.5 Итоговый датасет

В результате описанной процедуры был сформирован унифицированный обучающий набор данных, содержащий изображения животных и автоматически сгенерированные транспортно-значимые атрибуты. Такой подход позволил использовать открытые датасеты изображений животных и дополнить их необходимыми метаданными без ручной разметки.

Полученный датасет использовался для обучения мультимодальной модели, предсказывающей характеристики животного, влияющие на условия его транспортировки.

4.2 Описание модели

В системе используется Vision-Language модель Qwen2-VL-Instruct.

Модель принимает на вход изображение животного и текстовую инструкцию, описывающую формат требуемого ответа. В качестве результата модель генерирует JSON-объект с извлечёнными характеристиками.

Для повышения точности на задаче извлечения транспортно-значимых признаков была применена техника LoRA (Low-Rank Adaptation). Это позволяет дообучать крупные языковые модели с минимальным количеством обучаемых параметров.

Инференс модели выполняется при помощи фреймворка vLLM, обеспечивающего эффективную работу модели на GPU с ограниченным объёмом видеопамяти.

4.3 Процедура обучения модели

Для извлечения транспортно-значимых характеристик животных из изображений была использована мультимодальная модель Qwen2-VL-7B-Instruct. Обучение модели проводилось с использованием метода параметрически эффективной адаптации LoRA (Low-Rank Adaptation), что позволило существенно сократить число обучаемых параметров и снизить требования к вычислительным ресурсам.

4.3.1 Формирование обучающих примеров

Обучающий датасет был представлен в формате JSONL, где каждая запись содержит путь к изображению животного и соответствующую разметку транспортных характеристик. Для каждого изображения формировался целевой JSON-объект следующего вида:

```
{
 "size_class": "...",
 "weight_class": "...",
 "brachycephalic": true/false,
 "needs_carrier": "yes|no|unknown"
}
```

Каждый обучающий пример представлялся в виде диалога между пользователем и ассистентом. Пользовательское сообщение содержало изображение животного и текстовую инструкцию, описывающую формат требуемого ответа. Ответ ассистента представлял собой JSON-объект с извлечёнными характеристиками.

4.3.2 Предобработка изображений

Перед подачей в модель изображения приводились к фиксированному размеру. Использовался процессор Qwen2VLProcessor, который выполнял следующие операции:

- преобразование изображения в формат RGB;
- масштабирование изображения до размера 448×448 пикселей;
- нормализацию пиксельных значений.

Для ускорения загрузки данных была реализована система кэширования изображений, позволяющая хранить в памяти до 5000 уже загруженных изображений.

4.3.3 Формирование входных последовательностей

Для подготовки текстового входа использовался встроенный шаблон диалогов модели (chat template). Каждое сообщение преобразовывалось в текстовую последовательность, включающую как пользовательский запрос, так и ожидаемый ответ модели.

При обучении использовалась техника маскирования префикса (prefix masking). Токены, соответствующие пользовательскому запросу, исключались из функции потерь путём установки значения метки равным -100. Таким образом, модель обучалась предсказывать только целевой JSON-ответ.

4.3.4 Квантизация модели

Для уменьшения потребления видеопамяти применялась 4-битная квантизация весов модели с использованием библиотеки BitsAndBytes. Использовалась схема NF4 (Normal Float 4-bit) с двойной квантизацией.

Параметры квантизации:

- тип квантизации: NF4
- вычисления: float16
- использование двойной квантизации

Данный подход позволил обучать модель размером 7 миллиардов параметров на GPU с ограниченным объёмом памяти.

4.3.5 Заморозка визуального энкодера

Для повышения стабильности обучения параметры визуального энкодера модели были заморожены. Обучение проводилось только для языковой части модели и LoRA-адаптеров.

4.3.6 LoRA адаптация

Для дообучения модели применялся метод LoRA. Адаптеры добавлялись к проекционным матрицам механизма внимания трансформера.

Параметры LoRA:

- ранг адаптации $r = 16$
- коэффициент масштабирования $\alpha = 16$
- dropout = 0.05
- целевые слои: `q_proj`, `v_proj`

Использование LoRA позволило обучать лишь небольшое количество дополнительных параметров, сохранив основную модель неизменной.

4.3.7 Оптимизация и параметры обучения

Обучение проводилось с использованием оптимизатора AdamW. Основные параметры обучения представлены ниже:

- размер батча: 2
- накопление градиента: 4
- эффективный размер батча: 8
- число эпох: 2
- начальная скорость обучения: 5×10^{-5}
- weight decay: 0.01
- warmup: 5% шагов обучения
- смешанная точность: FP16

Для снижения потребления памяти использовалась техника gradient checkpointing.

4.3.8 Процедура обучения

Обучение проводилось с использованием класса Trainer из библиотеки HuggingFace Transformers. В процессе обучения выполнялась периодическая оценка модели на валидационном наборе данных. Лучшее состояние модели сохранялось на основе значения функции потерь на валидационной выборке.

В ходе обучения наблюдалось устойчивое снижение функции потерь. Начальное значение функции потерь составляло приблизительно 0.56, после чего оно быстро уменьшилось до уровня около 0.08. На валидационной выборке значение функции потерь стабилизировалось на уровне приблизительно 0.073.

Обучение проводилось на GPU NVIDIA RTX 3060 с 12GB видеопамяти.

Общее время обучения составило около 13 часов.

4.3.9 Результирующая модель

После завершения обучения LoRA-адаптеры были сохранены отдельно от базовой модели. При инференсе модель загружается совместно с обученными адаптерами и используется для генерации JSON-описания физических характеристик животного на основе входного изображения.

4.4 Архитектура предложенной модели

Архитектура разработанной системы состоит из следующих модулей:

1. Vision-Language модуль
2. Географический модуль
3. Модуль логических правил
4. Модуль оценки времени перевозки

4.4.1 Модуль анализа изображений

Модуль анализирует изображение животного и извлекает следующие характеристики:

- размер животного;
- весовая категория;
- признак брахицефалии;
- необходимость переноски.

4.4.2 Географический модуль

Географический модуль определяет:

- расстояние между пунктами маршрута;
- страну отправления и назначения;
- континенты маршрута;
- принадлежность маршрута одному континенту.

Расстояние вычисляется по координатам городов с использованием формулы гаверсинуса.

4.4.3 Модуль правил транспортировки

Модуль правил определяет допустимые способы транспортировки животного.

Рассматриваются следующие виды транспорта:

- автомобиль
- железнодорожный транспорт
- авиационный транспорт
- морской транспорт

Решение принимается на основе физических характеристик животного и географических ограничений маршрута в частности:

- возможность применения автомобильного или железнодорожного транспорта (начальная и конечная точка марштура - в пределах одного континента);
- возможность морской перевозки (страны начальной и конечной точки маршрута имеют выход к морю).

4.4.4 Модуль оценки времени перевозки

Для каждого допустимого вида транспорта вычисляется приблизительное время перевозки.

Используются следующие средние скорости:

- автомобиль: 70 км/ч
- поезд: 120 км/ч
- самолёт: 600 км/ч
- морской транспорт: 35 км/ч

4.5 Использование системы

Разработанная система запускается из командной строки и принимает три аргумента:

- путь к изображению животного;
- город отправления;
- город назначения.

Общий формат запуска программы:

```
python main_vlm.py <image_path> <origin_city> <destination_city>
```

Пример запуска системы:

```
python main_vlm.py "test_3.jpg" Berlin Tokyo
```

После выполнения программа анализирует изображение животного при помощи Vision-Language модели и определяет транспортно-значимые характеристики. Затем выполняется вычисление географических параметров маршрута и определяется список допустимых способов транспортировки.

Результатом работы программы является структурированный JSON-объект следующего вида:


```
{
  "physical_profile": {
    "size_class": "unknown",
    "weight_class": "unknown",
    "brachycephalic": false,
    "needs_carrier": false
  },
  "geo": {
    "distance_km": 8921.2,
    "origin_country": "Deutschland",
    "destination_country": "",
    "origin_country_code": "DE",
    "destination_country_code": "JP",
    "origin_continent": "Europe",
    "destination_continent": "Asia",
    "same_country": false,
    "same_continent": false
  },
  "transport_options": {
    "car": {
      "allowed": false,
      "mode": null,
      "eta_hours": null
    },
    "train": {
      "allowed": false,
      "mode": null,
      "eta_hours": null
    },
    "plane": {
      "allowed": true,
      "mode": "cargo",
      "eta_hours": 14.2
    },
    "sea": {
      "allowed": true,
      "mode": "containerized",
      "eta_hours": 254.9
```

```
    }
  }
}
```

Структура ответа состоит из трёх основных блоков.

4.5.1 Physical Profile

Блок `physical_profile` содержит характеристики животного, извлечённые из изображения:

- **size_class** — категория размера животного;
- **weight_class** — весовая категория;
- **brachycephalic** — признак брахицефалии;
- **needs_carrier** — необходимость использования переноски.

Эти параметры используются для определения допустимых способов перевозки.

4.5.2 Geographical Information

Блок `geo` содержит географические параметры маршрута:

- расстояние между городами;
- страны отправления и назначения;
- коды стран;
- континенты маршрута;
- принадлежность маршрута одной стране или одному континенту.

Эти данные используются для определения возможных транспортных маршрутов.

4.5.3 Transport Options

Блок `transport_options` содержит информацию о возможных способах перевозки:

- автомобиль;
- железнодорожный транспорт;
- авиационный транспорт;
- морской транспорт.

Для каждого вида транспорта указывается:

- допустимость использования (`allowed`);
- режим перевозки (`mode`);
- оценка времени в пути (`eta_hours`).

Если данный вид транспорта недоступен для указанного маршрута или типа животного, соответствующие поля принимают значение `null`.

Таким образом, система формирует единый структурированный ответ, который может быть использован в автоматизированных системах планирования перевозки животных.

5 Ограничения предложенного решения

Предложенная система имеет ряд ограничений. Во-первых, оценка времени перевозки основана на упрощённых средних скоростях транспорта и не учитывает реальные расписания, задержки и логистические ограничения. Во-вторых, используемая модель извлекает только ограниченный набор физических характеристик животного, что может быть недостаточно для точного моделирования всех условий транспортировки.

Кроме того, в данной работе не учитываются юридические и ветеринарные требования к перевозке животных, такие как наличие вакцинаций, карантинных ограничений и правил конкретных транспортных операторов. Эти аспекты рассматриваются как направление будущих исследований.

Также следует отметить ограничение, связанное с используемыми обучающими данными. Значительная часть обучающего датасета сформирована на основе Oxford-IIIT Pet Dataset, который содержит преимущественно изображения кошек и собак. В результате модель может демонстрировать более высокую точность при определении характеристик домашних животных по сравнению с другими видами (например, птицами, насекомыми или крупными сельскохозяйственными животными). Для повышения универсальности системы в дальнейшем планируется расширение обучающего набора данных за счёт включения дополнительных датасетов, содержащих более широкий спектр видов животных.

6 Заключение

В данной работе была разработана система планирования перевозки животных на основе анализа изображения и географических данных маршрута.

Основные результаты работы:

- разработана архитектура мультимодальной системы;
- реализован модуль извлечения характеристик животных из изображения;
- реализован модуль определения допустимых способов транспортировки;
- реализован модуль оценки времени перевозки.

Использование Vision-Language модели позволяет автоматизировать процесс определения параметров животного и упростить планирование перевозки.

В дальнейшем возможно расширение системы за счёт интеграции реальных транспортных правил авиакомпаний и железнодорожных операторов.

Предложенный подход демонстрирует, что мультимодальные языковые модели могут эффективно использоваться в задачах прикладной логистики, где требуется объединение визуального анализа и логического вывода.

Разработанный прототип подтверждает практическую применимость данного подхода и может служить основой для дальнейших исследований в области интеллектуальных систем планирования перевозок.

Исходный код разработанной системы доступен в открытом репозитории: `https://github.com/basil-77/animal-transport-llm`.

Список литературы

[1] Dennis Luxen and Christian Vetter. Real-time routing with openstreetmap data. *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011.

[2] OpenTripPlanner Contributors. Opentripplanner, 2023.

[3] MobilityData. General transit feed specification, 2024.

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.

[5] Aditya Khosla, Nityananda Jayadevaprakash, Bangpeng Yao, and Li Fei-Fei. Novel dataset for fine-grained image categorization. In *First Workshop on Fine-Grained Visual Categorization, IEEE CVPR*, 2011.

[6] Papers With Code. Animal image classification dataset, 2023.

[7] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. *International Conference on Machine Learning*, 2021.

[8] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023.

[9] Yichen Zhu, Minjie Zhu, Ning Liu, et al. Llava-phi: Efficient multi-modal assistant with small language model. *arXiv preprint arXiv:2401.02330*, 2024.

[10] Baichuan Zhou, Ying Hu, Xi Weng, et al. Tinyllava: A framework of small-scale large multimodal models. *arXiv preprint arXiv:2402.14289*, 2024.

[11] Shreya Rathod and Seema Patil. Pet classification using transfer learning. *International Journal for Research in Applied Science and Engineering Technology*, 2024.

[12] Omkar M. Parkhi, Andrea Vedaldi, and Andrew Zisserman. Cats and dogs. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.

[13] Kaggle. Animals-10 image dataset. `https://www.kaggle.com/datasets/alessiocorrado99/animals10`, 2019. Dataset containing images of 10 animal classes.